

Boosting and Bagging

March 2026

Contents

- 1 Bagging
- 2 Boosting
- 3 XGBoost
- 4 Оценочная функция
- 5 Отступление

$$a(x) = \frac{1}{k} (b_1(x) + \dots + b_k(x))$$

где $b_i(x) = b(x, X_i)$ — это базовая модель, а $a(x)$ — ансамбль, k — количество базовых моделей.

**** bootstrap** - генерации подвыборок с помощью семплирования с возвращением.

Сначала используем базовый алгоритм, например, дерево решений для регрессии. Затем ансамблевый метод, который создает множество копий базового алгоритма. Эти копии (N) будут обучаться независимо на разных подвыборках данных. Предсказание усредняем: таким образом variance будет в N раз ниже нежели в отдельности. Когда количество элементов ансамбля становится достаточно(!) большим, то дальнейшее увеличение не увеличивает качество.

Все участники ансамбля обучаются последовательно друг другу, таким образом, исправляя ошибки предыдущего. С какого-то момента во время увеличения N начнется переобучение. Во избежание такого можно выбирать очень простую базовую модель, либо уменьшать вес последующих алгоритмов в композиции.

$$a_N(x) = \sum_{n=0}^N \eta \gamma_n b_n(x)$$

где η — параметр шага (learning rate). Именно она будет контролировать влияние новых компонент ансамбля

a — итоговое предсказание

b_n — базовый алгоритм n -й (например, дерево решений)

γ_n — вес n -го алгоритма

N — количество алгоритмов в ансамбле

Такой подход будет медленнее чем bagging, но зато склонность итоговой модели к переобучению будет меньше.

- 1 Внутри одного дерева поиск решения по разным признакам идет параллельно.
- 2 Автоматически обрабатывает NaN.
- 3 Добавляются штрафы в функцию потерь, например, за добавление нового листа, за большие веса в листьях делает веса разреженными(=0)

Regularized Learning Objective (XGBoost):

$$\mathcal{L} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{t=1}^T \left(\gamma |\text{leaf}_t| + \frac{1}{2} \lambda \sum_{j=1}^{|\text{leaf}_t|} w_j^2 + \alpha \sum_{j=1}^{|\text{leaf}_t|} |w_j| \right)$$

- 4 Градиентный бустинг использует только первую производную, здесь же разложение Тейлора до второго порядка.

1

$$L \approx L(y, \hat{y}) + g \cdot f_t(x)$$

2

$$L \approx L(y, \hat{y}) + g \cdot f_t(x) + \frac{1}{2} h \cdot f_t(x)^2$$

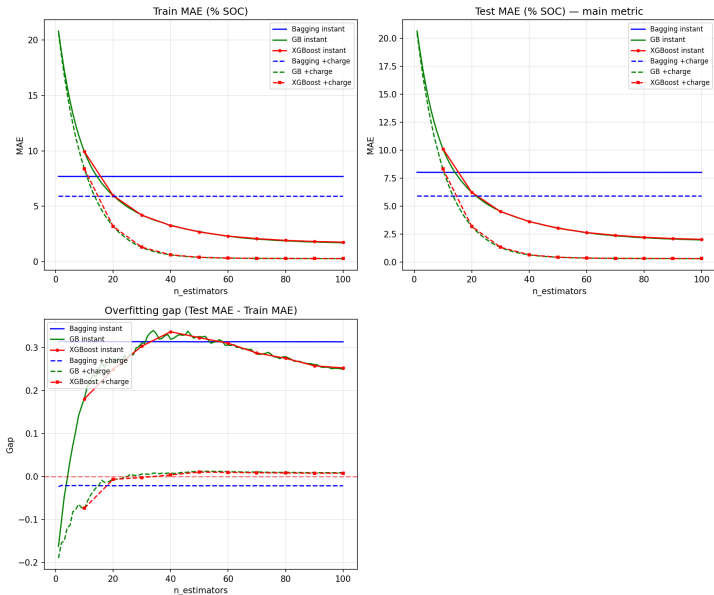
$$Q(a) = \mathbb{E}_x \left[\text{bias}_X^2 a(x, X) \right] + \mathbb{E}_x [V_X[a(x, X)]] + \sigma^2$$

Раскладываем на шум, смещение и разброс. С шумом ничего сделать не получится. Смещение композиции равно смещению одного алгоритма. Дисперсия композиции в k раз меньше дисперсии отдельного алгоритма (если базовые алгоритмы не коррелированы)

Возможно идея считать заряд оказалась неудачной. The reference SOC values are already calculated using coulomb counting for convenience. [

$$SOC(t) \approx SOC(t_0) + \frac{1}{C} \int_{t_0}^t I(\tau) d\tau]$$

BG, XGB, Bagging(V, I, T, (+Charge))



BG, XGB, Bagging (only Charge)

